

Documentation: TCP Command Server on Raspberry Pi

Intern Name: Niharika M

Project Goal: To build a network-based command server using a Raspberry Pi, allowing it to listen for TCP connections and perform actions like simulating sensor data, reporting CPU temperature, and processing virtual commands like LED_ON.

Use Case

Enable **remote interaction with a Raspberry Pi** via a TCP client from any device on the same network. The goal is to:

- **Simulate hardware behavior** without actual GPIO use.
- Provide **real-time command-response communication** over the network.
- Support **multi-device access** (e.g., from Windows, another Pi, or Linux systems) using simple Python scripts.

System Setup

Platform: Raspberry Pi OS (Bookworm)

Language: Python 3

Networking: TCP/IP Protocol

Client Device: Windows laptop (used to send commands)

What is TCP?

- **TCP (Transmission Control Protocol)** is a **reliable, connection-oriented protocol** used in network communication.
- Ensures **data arrives in order and without errors**.
- Often used for applications like web traffic (HTTP), email, and file transfers.

Comparison with other protocols:

Protocol	Type	Features	Use Case
TCP	Connection-oriented	Reliable, ordered, error-checked transmission	HTTP, FTP, SSH
UDP	Connectionless	Faster, no guarantee of delivery/order	Live streaming, DNS, VoIP
HTTP	Application Layer	Built on TCP, used for web communication	Browsers, APIs

How It Works

1. **Raspberry Pi** runs a TCP server that listens for incoming connections on a specific port (e.g., 12345).
2. **Client** connects using the Pi’s IP and sends text-based commands.
3. **Pi parses the command** and responds appropriately (e.g., sends CPU temperature or simulates turning on an LED).
4. Responses are sent **back over the same connection**.

Commands Implemented

Command	Action
TEMP	Returns the current CPU temperature of the Pi
LED_ON	Simulates turning on an LED
LED_OFF	Simulates turning off an LED
SENSOR	Returns simulated sensor values (e.g., humidity)
EXIT	Closes the client connection

Code Snapshot

tcp_server.py (on Raspberry Pi):

```
python
import socket, random, psutil

def get_temp():
    try:
        with open("/sys/class/thermal/thermal_zone0/temp", "r") as f:
            return f"temp={float(f.read())/1000:.1f}'C"
    except:
        return f"temp={psutil.sensors_temperatures()['cpu-thermal'][0].current:.1f}'C"

HOST, PORT = '0.0.0.0', 12345
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)
print(f"[+] Listening on {HOST}:{PORT}")

while True:
    client, addr = server.accept()
    print(f"[+] Connected by {addr}")
    while True:
        data = client.recv(1024).decode().strip()
        if not data or data.upper() == "EXIT":
            break
        elif data.upper() == "TEMP":
            client.send(get_temp().encode())
        elif data.upper() == "LED_ON":
            client.send(b"LED turned ON (simulated)")
        elif data.upper() == "LED_OFF":
            client.send(b"LED turned OFF (simulated)")
        elif data.upper() == "SENSOR":
            fake_data = f"humidity={random.randint(30,60)}%, light={random.randint(100,500)}lx"
            client.send(fake_data.encode())
        else:
            client.send(b"Unknown command")
    client.close()
```

Client Example (Windows)

tcp_client.py:

```
python
import socket

HOST = '172.16.xx.xx' # Use your RPi's IP
PORT = 12345
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

while True:
    cmd = input("Send command to Raspberry Pi (TEMP, LED_ON, SENSOR, EXIT): ")
    client.send(cmd.encode())
    if cmd.upper() == "EXIT":
        break
    print("Response from Raspberry Pi:", client.recv(1024).decode())

client.close()
```

Working Results

- **Command: TEMP** → Response from Raspberry Pi: temp=46.7'C
- **Command: LED_ON** → Response: LED turned ON (simulated)
- **Command: SENSOR** → humidity=42%, light=371lx

Learnings

- TCP provides **reliable communication** ideal for command-and-control scenarios.
- A **simple Python script** is sufficient to implement and test custom TCP servers.
- Understanding IP addresses, ports, and firewalls is key to successful connections.
- Important to **allow the port (12345)** through Raspberry Pi's firewall using `sudo ufw allow 12345`.

FileEditSelectionViewGoRun...<-->Search

tcp_client.py X

D: > Desktop > tcp_client.py > ...

```
1 import socket
2
3 HOST = '172.16.38.8' # Replace with your Raspberry Pi IP
4 PORT = 12345
5
6 with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
7     client.connect((HOST, PORT))
8     client.sendall(b'CPU_TEMP') # You can change this command to test
9     data = client.recv(1024)
10
11 print('Response from Raspberry Pi:', data.decode())
12
13
14
15
16
17
18
```

RUN AND DEBUG

Run and Debug

To customize Run and Debug, open a folder and create a launch.json file.

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

File "D:\Desktop\tcp_client.py", line 7, in <module>
client.connect((HOST, PORT))
ConnectionRefusedError: [WinError 10061] No connection could be made because the target machine actively refused it

PS D:\Desktop> ^C
PS D:\Desktop>
PS D:\Desktop> d:; cd 'd:\Desktop'; & 'c:\Users\Niharika M\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\Niharika M\.vscode\extensions\ms-python.debugpy-2025.10.0-win32-x64\bundled\libs\debugpy\launcher' '53532' '--' 'D:\Desktop\tcp_client.py'
Response from Raspberry Pi: temp=47.7'C
PS D:\Desktop>

powersh...
Python Deb...

BREAKPOINTS
☐ Raised Exceptions
☒ Uncaught Exceptions
☐ User Uncaught Exceptions

Ln 7, Col 5Spaces: 4UTF-8CRLF()Python3.11.9 (Microsoft Store)